

Basilisk Installation Guide on Windows



A complete step-by-step video version of this installation guide is included in the folder named "Video_Guide".

1.Introduction

Welcome to the Basilisk Installation Guide on Windows Operating System. This document provides a comprehensive, step-by-step walkthrough for installing and configuring the Basilisk simulation framework (version 2.2.1) on a Windows 10/11 machine. All instructions reflect the setup tested on Windows as of March 9, 2024, and are based on the official Basilisk documentation and GitHub resources. Whether you are new to Basilisk or upgrading from an earlier release, this guide will help ensure that your environment is correctly prepared and that the core components (Basilisk and Vizard) build and run without errors.

Requirements overview (hardware, OS version)

Before proceeding, please verify that your system meets the following prerequisites:

- **Operating System:** Windows 10 or Windows 11 (64-bit).
- **Hardware:** A multicore processor (Intel i5 or higher recommended), at least 8 GB of RAM, and ≥ 20 GB of free disk space.
- **Administrative Privileges:** You must be able to run Command Prompt (or PowerShell) as an administrator for certain installation steps (e.g., installing Visual Studio and running Conan).
- **Internet Access:** Required for downloading installers, Python packages, and Conan dependencies.

Software Requirements

- **CMake (≥ 3.14):** Used to generate build files. Tested version: 3.29.0-rc2.
- **Python ($\geq 3.8.x$):** Basilisk's Python interface requires Python 3. We recommend using Python 3.11.7. Ensure that "pip" is selected during installation.
- **pip:** Python's package installer. Confirm it was enabled at install time.
- **Visual Studio (15 2017 / 17 2022 or newer):** Install the "Desktop development with C++" workload.
- **SWIG (version 3.x or 4.1; do not use 4.2):** Required to generate Python bindings. Download and unzip SWIG (e.g., to C:\Program Files\Swig) so that swig.exe is reachable via your PATH.

Once you have confirmed that all of the above software and system requirements are in place, you are ready to move on to the detailed installation steps, beginning with setting up your environment variables and installing the necessary tools.

2. Software Setup

2.1 Install CMake

Download CMake

- Visit the official CMake website: <https://cmake.org/>
- Download a Windows installer for CMake version 3.14 or higher (the guide was tested on 3.29.0-rc2).

Run the Installer

- Launch the downloaded `.msi` and follow the prompts.
- When asked to select components, keep the defaults (including “Add CMake to the system PATH for all users” if offered).

Verify Installation

- Open a new Command Prompt (press **Win + R**, type `cmd`, and press **Enter**).
- Type: `cmake --version`
- You should see output similar to **cmake version 3.29.0-rc2**
- If you receive an error (e.g., “cmake is not recognized”), double-check that `C:\Program Files\CMake\bin` is on your PATH

2.2. Install Python

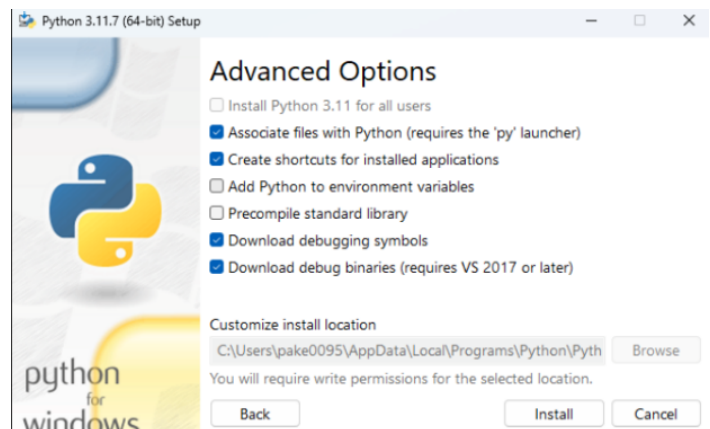
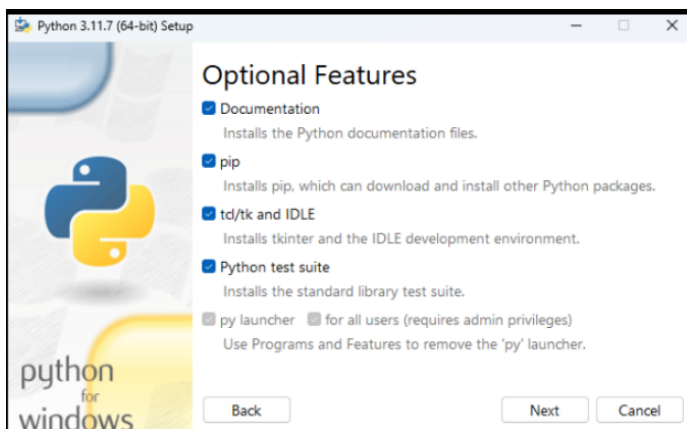
Download Python

- Go to the official Python downloads page:
<https://www.python.org/downloads/windows/>
- Choose a Windows installer for Python 3.8.x or newer (this guide uses Python 3.11.7).

Run the Python Installer

- Launch the `.exe` installer.

- IMPORTANT: On the first screen, check the box “Add Python 3.x to PATH.”
- Click “Customize installation.”
- In the next screen, ensure “pip”, “tk/tk and IDLE,” and “Documentation” are selected.
- Click “Next.”
- In the Advanced Options screen, check both:
 - “Download debugging symbols”
 - “Download debug binaries (requires VS 2017 or later).”
- Leave other defaults, and click “Install.”
- Wait for installation to complete, then click “Close.”



2.3. Install pip

Verify Python & pip

- Open a new Command Prompt window.
- Type: **python -version**
- You should see something like: **Python 3.11.7**
- Next, verify pip: **pip -version**
- If either command fails, double-check that the installer added Python to your system PATH.

Confirm pip Installation

Note: If you selected “pip” during Python setup, no additional steps are needed here. However, we recommend running the following to ensure everything’s in order.

- Upgrade pip (Optional but Recommended): **python -m pip install --upgrade pip**
- Test pip by Installing a Small Package: **pip install --upgrade setuptools**
- Check pip List: **pip list**
- Verify that setuptools and its dependencies appear in the list, this confirms that pip works correctly.

```
(basilisk-env) PS C:\Users\perer\OneDrive\Desktop\Swim 2025 sem 1\project A\satelliteConstellation-basilisk\ProjectWork\BskSim\core> pip list
```

Package	Version	Editable project location
abs1-py	2.2.2	
aiofiles	24.1.0	
annotated-types	0.7.0	
anyio	4.9.0	
argon2-cffi	23.1.0	
argon2-cffi-bindings	21.2.0	
arrow	1.3.0	
astroid	2.11.7	
asttokens	3.0.0	
astunparse	1.6.3	

2.4. Install Visual Studio (C++)

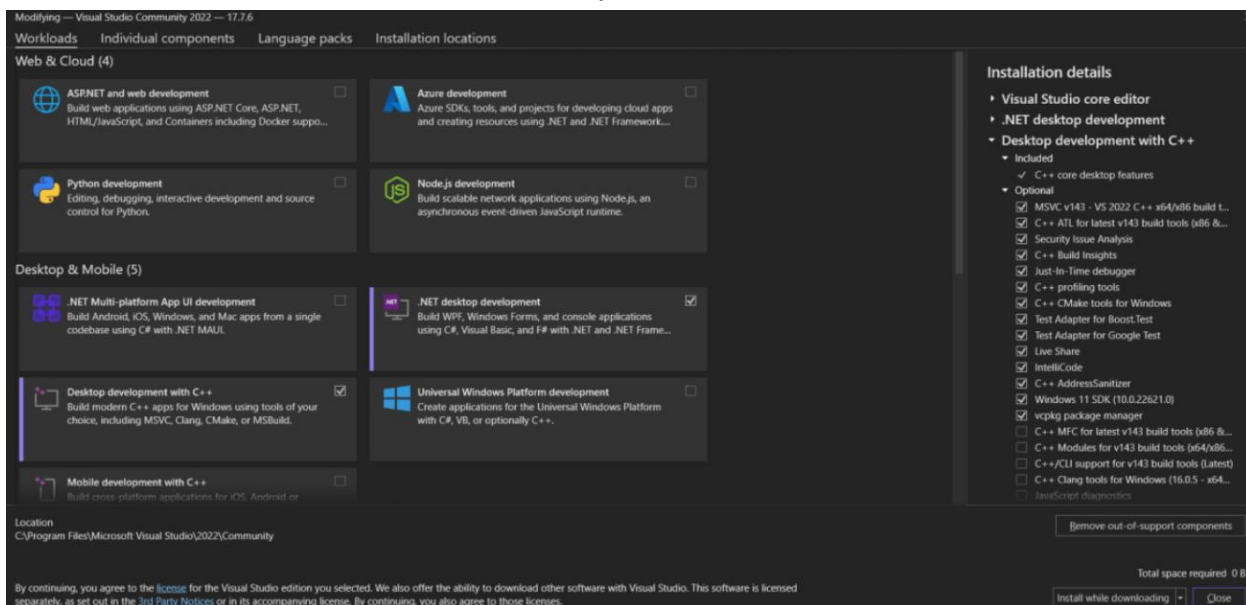
Download Visual Studio Installer

- Go to <https://visualstudio.microsoft.com/downloads/>
- Download “Visual Studio 2022 Community”.

Run the Visual Studio Installer

- Launch the downloaded installer (vs_community.exe).
- In the “Workloads” tab, select “Desktop development with C++.” This ensures you have the MSVC compiler toolchain and Windows SDK.
- Optionally, under “Installation details” on the right, you can include:
 - “C++ CMake tools for Windows” (if it isn’t already implied by the workload).
 - “Windows 10 SDK” (10.0.x) (usually bundled).

- Click “Install” and wait for the components to download and install.



2.5. Install SWIG

Basilisk uses SWIG to generate Python bindings for its C++ modules.

Download SWIG

- Visit <http://www.swig.org/download.html>
- Under the “Windows” section, download “**swigwin-4.1.1.zip**” (or any SWIG 4.1.x).
- **DO NOT** use SWIG 4.2, as it is known to cause build issues on Windows.

Unzip SWIG

- Extract the contents of the ZIP (which contains swig.exe and support files) to a permanent location, e.g.: **C:\Program Files\Swig**
- The final path should contain swig.exe, e.g.: **C:\Program Files\Swig\swig.exe**

Verify SWIG Installation

- Open a new Command Prompt.
- Type: **swig -version**

- You should see output similar to: **SWIG Version 4.1.1**

If the command fails, you will add SWIG's directory to your PATH in the next subsection.

2.6. Configure Environment Variables

Several tools installed above need to be discoverable via the command line. This subsection walks you through setting system (or user) environment variables so that cmake, swig, and later Basilisk binaries can be found.

Important: Whenever you modify environment variables, you must close and reopen any open Command Prompt windows and sometimes reboot to ensure changes take effect.

Open Environment Variables Panel

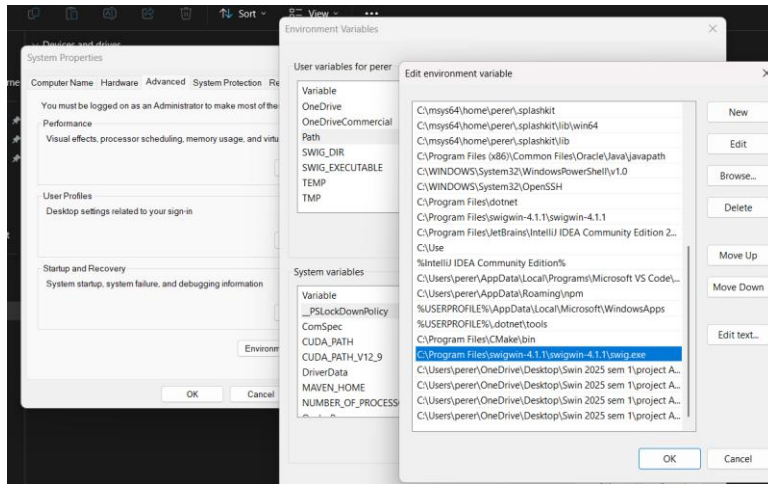
- Right-click **This PC** (or **My Computer**) on your Desktop or in File Explorer, then click **"Properties."**
- In the System window, click **"Advanced system settings"** on the left.
- In the "System Properties" dialog, under the **Advanced** tab, click **"Environment Variables..."**

Add CMake to PATH (if not already added)

- In the **User variables** (or **System variables**) section, locate and select **"Path,"** then click **"Edit."**
- Click **"New"** and enter (or verify) the folder where cmake.exe lives. By default, that is: **C:\Program Files\CMake\bin**
- Click **"OK"** to save.

Add SWIG to PATH

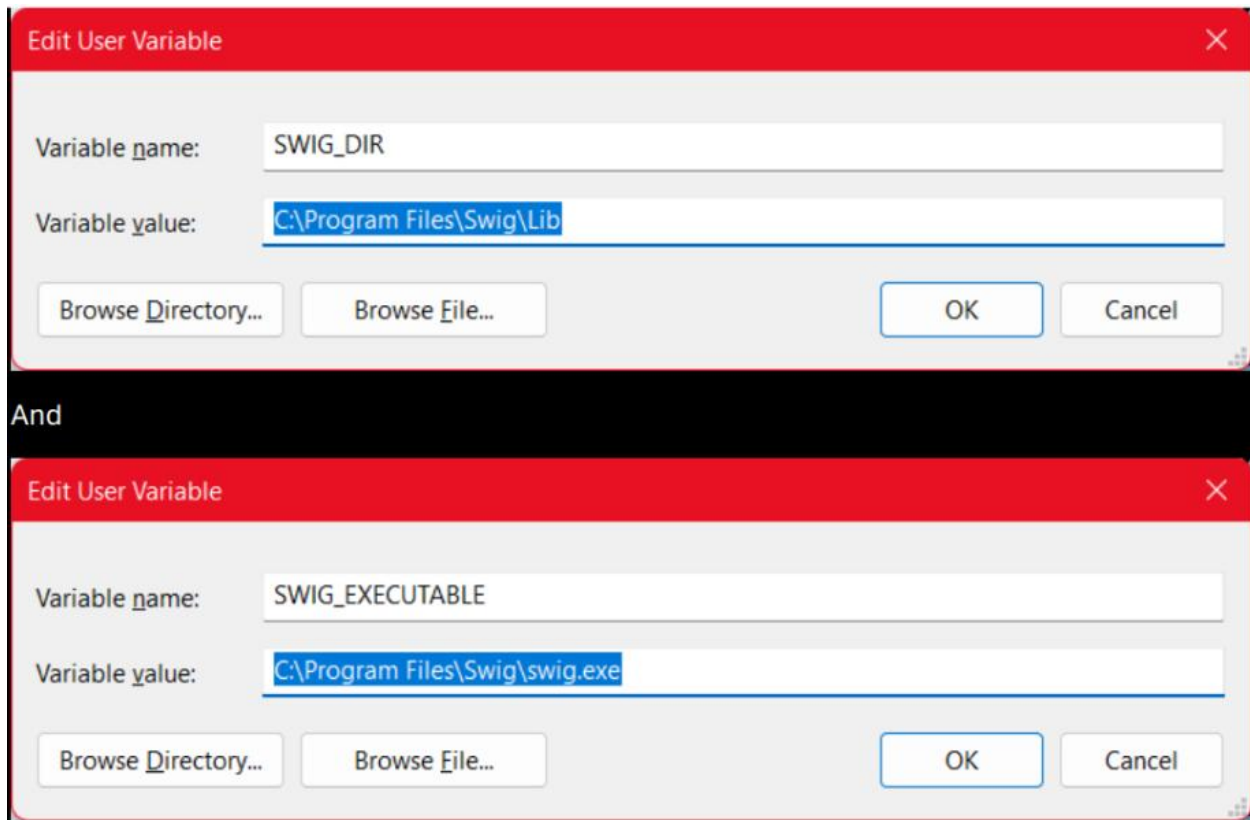
- In the same **Path** editor (under User or System variables), click **"New"** and add: **C:\Program Files\Swig**
- Click **"OK."**



Create SWIG_DIR and SWIG_EXECUTABLE Variables

- Under **User variables** (or **System variables**), click **“New.”**
 - For **Variable name**, enter: **SWIG_DIR**
 - For **Variable value**, enter the swig root folder (where swig.exe resides), e.g.:
C:\Program Files\Swig
- Click **“OK.”**

- Again, click **“New.”**
 - For **Variable name**, enter: **SWIG_EXECUTABLE**
 - For **Variable value**, enter the full path to swig.exe, e.g.: **C:\Program Files\Swig\swig.exe**
- Click **“OK.”**



3. Download Basilisk

Instead of downloading a ZIP archive, Basilisk must be cloned using Git to ensure all submodules and version information are correctly retrieved. Follow the steps below to clone the v2.2.1

3.2. Clone the Basilisk Repository

Open Command Prompt

- Navigate to the directory where you want to keep the Basilisk source (e.g., C:\Dev\, D:\Projects\, etc.):

Clone the Repository

- Run: git clone <https://github.com/AVSLab/basilisk.git>
- This creates a folder named basilisk.

Note: Go to <https://github.com/AVSLab/basilisk/tags> and find the correct basilisk version(2.2.1).

4. Python Environment Setup

This section guides you through creating an isolated Python environment for Basilisk, installing all required packages, and verifying the installation.

4.1. Install virtualenv

Open a Command Prompt

- Press Win + R, type cmd, and press Enter.
- Navigate to your Basilisk root directory (e.g., where you cloned basilisk)

Install virtualenv Globally

- Run: **pip install virtualenv**
- Wait for pip to download and install virtualenv.

4.2. Create a Virtual Environment

- From the Basilisk Root Folder, create a new virtual environment named basilisk-env by entering: **virtualenv basilisk-env**
- This command creates a basilisk-env subfolder containing an isolated Python interpreter and package directory.

Confirm Creation

- Verify that a new folder named basilisk-env exists alongside folders such as basilisk, src, and examples.

Activate the Virtual Environment

- In your Command Prompt (still in C:\Dev\basilisk), run: **.\basilisk-env\Scripts\activate.bat**

- After activation, your prompt should prepend with (basilisk-env), indicating all python and pip commands now use this isolated environment.

4.3. Install Required Python Packages

- Within the Activated Environment ((basilisk-env)), install Basilisk's core Python dependencies by running: **pip install pandas numpy matplotlib pytest Pillow conan==1.63.0 parse>=1.18.0**
- **pip install -r requirements-win.txt**
- Additional dependencies (e.g., scipy, networkx, pyyaml) will be fetched automatically when you build Basilisk with Conan.

4.4. Verify Installed Packages

List Installed Packages

- Still in (basilisk-env), run: **pip list**
- Confirm that at least the following entries appear:

```
astroid==2.11.7      isort==5.13.2      pluginbase==0.7
bottle==0.12.25      Jinja2==3.1.3      Pygments==2.17.2
certifi==2024.2.2    kiwisolver==1.4.5  PyJWT==2.8.0
charset-normalizer==3.3.2 lazy-object-proxy==1.10.0 pylint==2.14.2
colorama==0.4.6      MarkupSafe==2.0.1  pyparsing==3.1.1
conan==1.63.0         matplotlib==3.8.3  pytest==8.0.2
contourpy==1.2.0     mccabe==0.7.0      python-dateutil==2.9.0.post0
cyclerc==0.12.1      node-semver==0.6.1  tz==2024.1
deprecation==2.0.7   numpy==1.26.4      PyYAML==6.0.1
dill==0.3.8          packaging==23.2    requests==2.31.0
distro==1.1.0         pandas==2.2.1      six==1.16.0
eigen==0.0.1          parse==1.20.1      tomlkit==0.12.4
fasteners==0.19      patch==1.16         tqdm==4.66.2
fonttools==4.49.0    patch-ng==1.17.4   tzdata==2024.1
future==0.16.0        pillow==10.2.0     urllib3==1.26.18
idna==3.6             platformdirs==4.2.0 wrapt==1.16.0
iniconfig==2.0.0     pluggy==1.4.0
```

- Run pip list and compare against the requirements.txt list. If some packages are missing you have to install them manually.

5. Build Basilisk with Conan

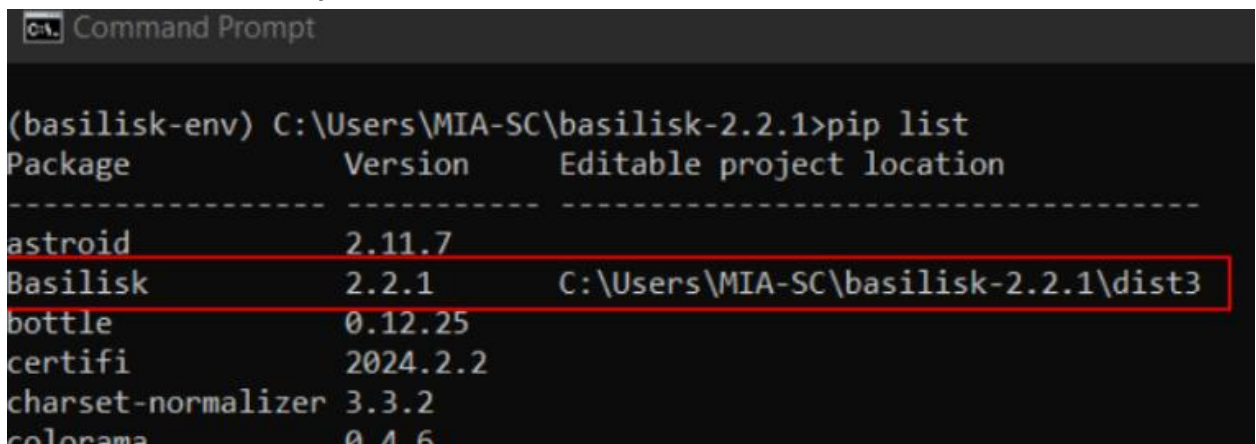
5.2. Run Conan Install

Note: You have to close all the terminals and reopen it with activated environment

- Click Start, type “cmd”, then right-click “Command Prompt” and select “Run as administrator.”
- Execute the Conan build command: **python conanfile.py --generator "Visual Studio 17 2022"**
- **Note:** The build can take **10–15 minutes** on typical hardware. Remain patient and watch for any error messages.

5.3. Verify Basilisk Package

- After Conan completes without errors, open a **new** (non-Administrator) Command Prompt.
- Reactivate your virtual environment and return to the Basilisk root.
- List installed Python packages to confirm that **basilisk** appears.
- You should see an entry similar to:

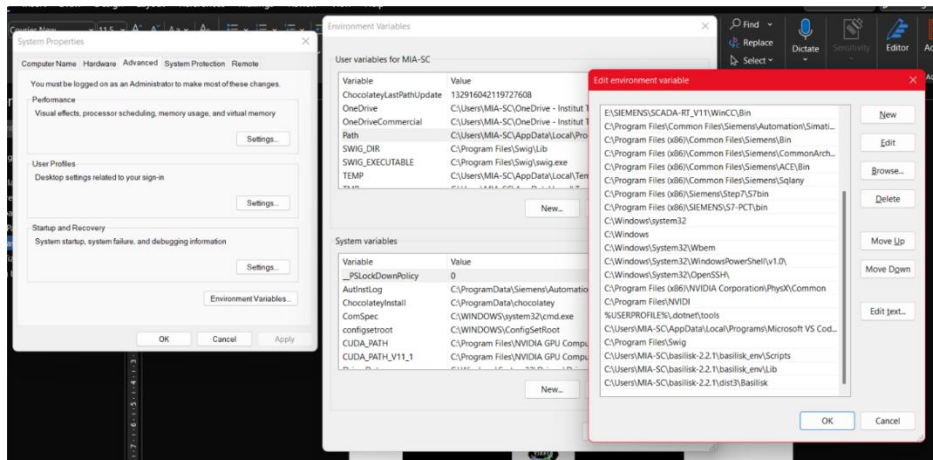


```
Command Prompt

(basilisk-env) C:\Users\MIA-SC\basilisk-2.2.1>pip list
Package            Version            Editable project location
-----
astroid            2.11.7
Basilisk            2.2.1              C:\Users\MIA-SC\basilisk-2.2.1\dist3
bottle             0.12.25
certifi            2024.2.2
charset-normalizer 3.3.2
colorama           0.4.6
```

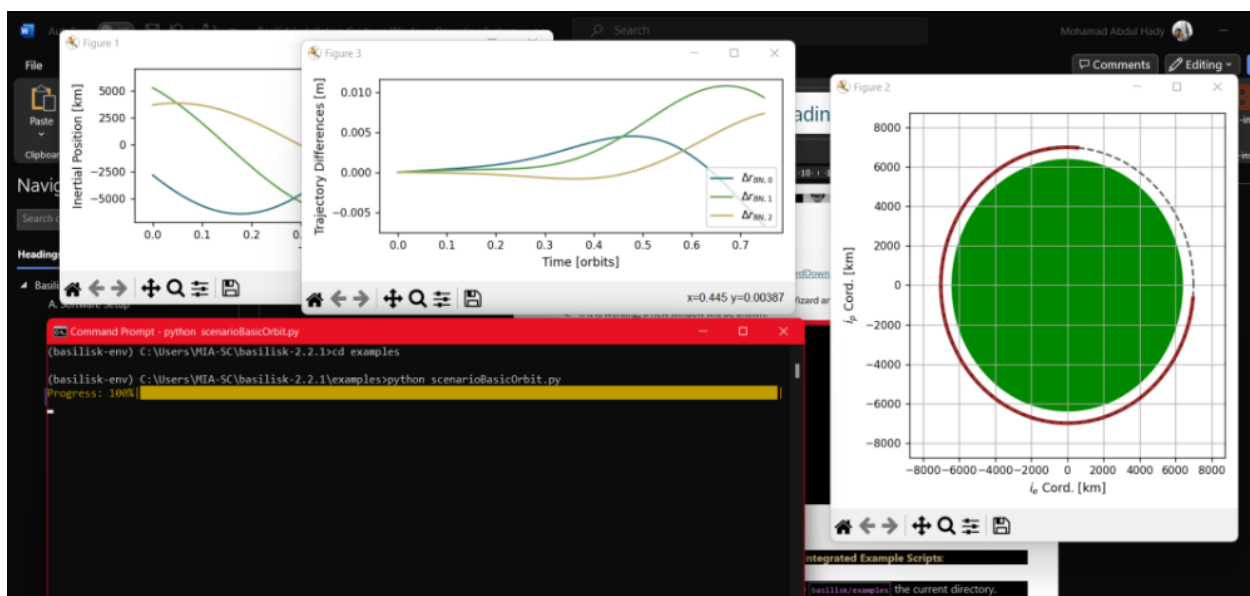
- Confirm that a new dist3 folder exists inside your Basilisk root.
- The presence of **dist3\Basilisk** indicates that the C++ library and Python module were built and staged correctly.

- Add a new environment variable path: C:\Users\MIA-SC\basilisk-2.2.1\dist3\Basilisk



6. Run basic Basilisk example code

- First, open a Command Prompt and navigate to your Basilisk installation directory. Activate the virtual environment you created earlier by running.
- Once activated, change into the examples subfolder: **cd examples**
- To execute a simple, non-visual example that generates orbital plots, enter: **python scenarioBasicOrbit.py**
- This script will produce a series of Matplotlib windows displaying various orbital state variables (e.g., position, attitude). Simply close the plot windows when you are finished reviewing them.



7. Install Vizard for 3D Visualization

Vizard provides real-time 3D visualization of Basilisk simulations. Follow the steps below to download, extract, and verify Vizard on Windows.

7.1. Download Vizard

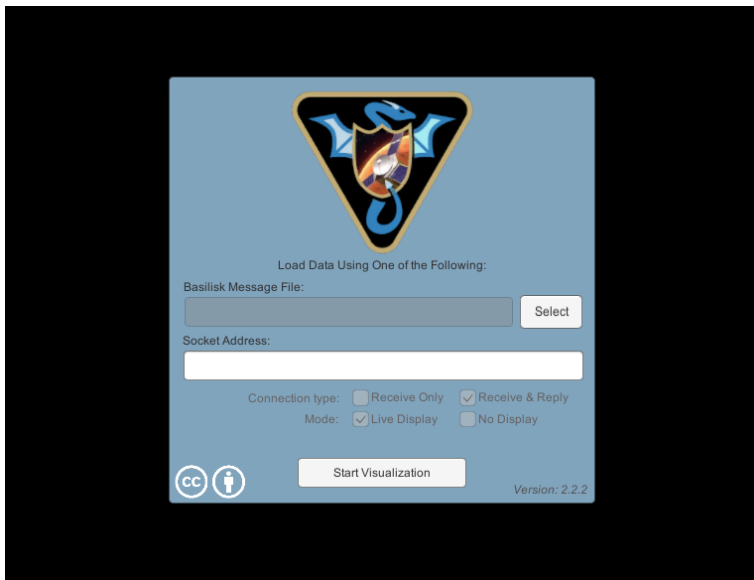
Open Your Web Browser and navigate to:

<https://hanspeterschaub.info/basilisk/Vizard/VizardDownload.html>

Download the ZIP Archive to a folder of your choice

Extract and Run Vizard

- Navigate to the Download Location in File Explorer (e.g., C:\Downloads).
- Right-Click the ZIP Archive (Vizard-1.0-win.zip) and select **“Extract All...”**.
- Inside the extracted Vizard folder, go to Windows\Vizard and run the Vizard.exe file
- If it is working, a new window will be shown



7.2. Run the example of VIZARD live visualization:

- Go to Basilisk example directory: `cd \basilisk-2.2.1\examples`
- Run the example: `python scenarioBasicOrbitStream.py`
- Copy the tcp address: **tcp://localhost:5556**
- Paste to Vizard socket address:

8. Installing BSK-RL (Basilisk Reinforcement Learning Framework)

BSK-RL provides a reinforcement learning extension for Basilisk, enabling the simulation of autonomous spacecraft control and training environments. The following steps explain how to correctly install and verify BSK-RL version 1.1.0 to work with Basilisk 2.2.1.

8.1. Clone the BSK-RL Repository

Open a new terminal with your **Basilisk environment activated** and run outside of the basilisk folder:

```
git clone -b v1.1.0 https://github.com/AVSLab/bsk\_rl.git  
cd bsk_rl
```

8.2. Install BSK-RL

Within the same activated environment, install BSK-RL using: **pip install -e .**

This links the BSK-RL package directly to your current virtual environment.

8.3. Verify Installation

To confirm that BSK-RL is installed, run: **pip show bsk_rl**

If successful, you will see details such as:

```
(basilisk-env) C:\Users\perer\Desktop\Swin_sem2\ProjectB\basilisk-2.2.1>pip show bsk_rl  
Name: bsk_rl  
Version: 1.1.0  
Summary: RL environments and tools for spacecraft autonomy research, built on Basilisk. Developed by the AVS Lab.
```

8.4. Run Compatibility Tests

To ensure BSK-RL is correctly connected to Basilisk, execute: **pytest tests/unittest**

All unit tests should be completed successfully. Some warnings are normal, but there should be no import errors (e.g., No module named 'Basilisk').

```
(basilisk-env) C:\Users\perer\Desktop\Basilisk\bsk_rl>pytest tests/unittest
===== test session starts =====
platform win32 -- Python 3.11.7, pytest-7.3.1, pluggy-1.5.0
rootdir: C:\Users\perer\Desktop\Basilisk\bsk_rl
plugins: cov-6.1.1, repeat-0.9.4
collected 408 items

tests\unittest\test_gym_env.py ..... [ 9%]
..... [ 12%]
tests\unittest\act\test_actions.py ..... [ 16%]
tests\unittest\comm\test_communication.py ..... [ 20%]
tests\unittest\data\test_composition.py ..... [ 24%]
tests\unittest\data\test_data.py ..... [ 32%]
. [ 33%]
tests\unittest\obs\test_observations.py ..... [ 38%]
tests\unittest\sats\test_access_satellite.py ..... [ 44%]
.....S..... [ 54%]
tests\unittest\sats\test_satellite.py ..... [ 58%]
tests\unittest\scene\test_scenario.py ..... [ 65%]
.. [ 66%]
tests\unittest\sim\test_dynamics.py ..... [ 74%]
..... [ 77%]
tests\unittest\sim\test_fsw.py ..... [ 79%]
tests\unittest\sim\test_simulator.py ..... [ 81%]
tests\unittest\sim\test_world.py ..... [ 84%]
tests\unittest\utils\test_functional.py ..... [ 89%]
```


9. Cloning and Running the Project GUI

9.1. Cloning the Project Repository

It is important to install or clone this project in the same directory where your Basilisk folder is located, but not inside the Basilisk folder itself. For example, both folders should exist side-by-side in the same main directory.

To set up the complete project, including all simulation scripts and the graphical user interface (GUI), clone the repository using the following command in your terminal:

```
git clone CompProjectA/satelliteConstellation-basilisk
```

This will download the full project containing:

- The main simulation modules
- The BskSim/core folder with the GUI launcher
- Fault injection and configuration scripts

9.2. Running the GUI

Once the project and environment are set up:

1. Open the project folder and navigate to the location of Run_Fault_Simulator.bat.
2. Double-click the .bat file to launch the Fault Simulator GUI.
3. The script will automatically:
 - Activate your Basilisk virtual environment
 - Execute the run_fault_simulator.py file
 - Keep the command window open for logs and messages

If the environment and project were installed correctly, the GUI will launch without errors.

9.3. If Installed in a Different Location

If the project or Basilisk folder is installed in a different path, you will need to update the environment path inside the .bat file.

1. Right-click the file GUI.bat and choose Edit.

2. Locate the line that activates the environment, for example:
call "..\..\..\basilisk-2.2.1\basilisk-env\Scripts\activate.bat"
3. Replace it with the full path to your local environment's activate.bat file, for example:
call "D:\Projects\basilisk-2.2.1\basilisk-env\Scripts\activate.bat"
4. Save the file and double-click it again to relaunch the GUI.

Troubleshooting & Tips

Common build errors (e.g., missing environment variables, incompatible SWIG version)

Verifying the correct Python interpreter and virtual env activation

Appendices

- 1). Full list of required Python packages (requirements.txt)